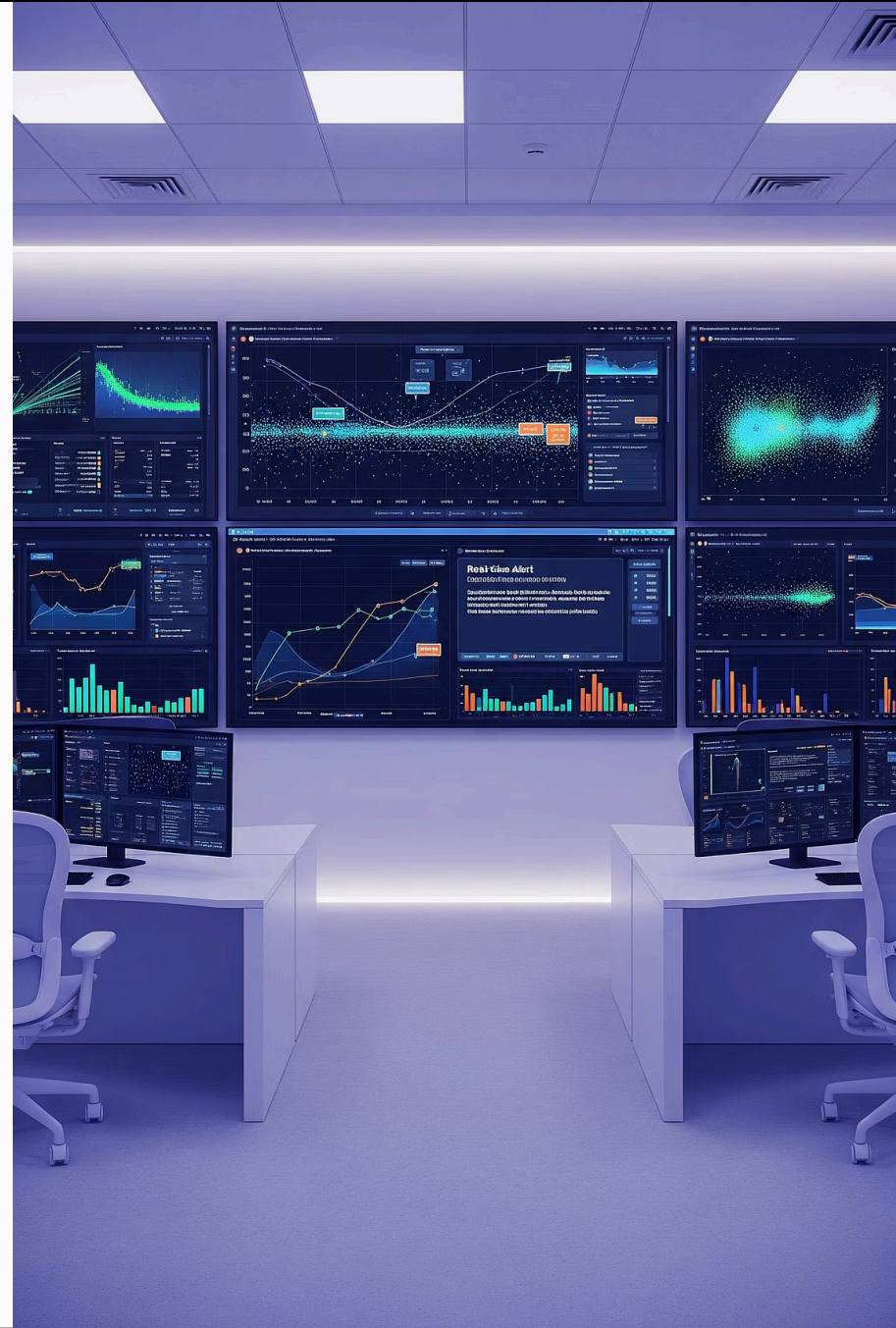


Drift Detection Pipeline: From Signals → Actions

Detect **data drift** & **concept drift** before users do with batch + streaming architectures using tools like Evidently, River/Alibi-Detect, Prometheus, Airflow/Flink, and Feature Store



Why Drift Detection Matters



Silent Accuracy Decay

Models degrade without visible errors, leading to poor user experience and revenue loss



Regulatory Risk

Compliance concerns in regulated industries like finance, healthcare, and insurance



Cost Waste

Resources wasted retraining models when the wrong component is failing



Safer Operations

Early detection enables safer rollouts and significantly reduces on-call incidents



Taxonomy & Signals

Numerical

Mean/variance shifts, distributional tests comparing reference vs. current data

Categorical

Proportion changes across categories, sudden spikes in rare categories

Embeddings

Centroid/covariance shift, cosine distance, Earth Mover's Distance (EMD)

LLMs

Prompt mix shift, retrieval hit-rate drift in RAG systems, toxicity/safety drift

Core Metrics by Data Type

Numerical Features

- Kolmogorov-Smirnov test
- Population Stability Index (PSI)
- Jensen-Shannon/Kullback-Leibler divergence
- Maximum Mean Discrepancy (MMD)

Categorical Features

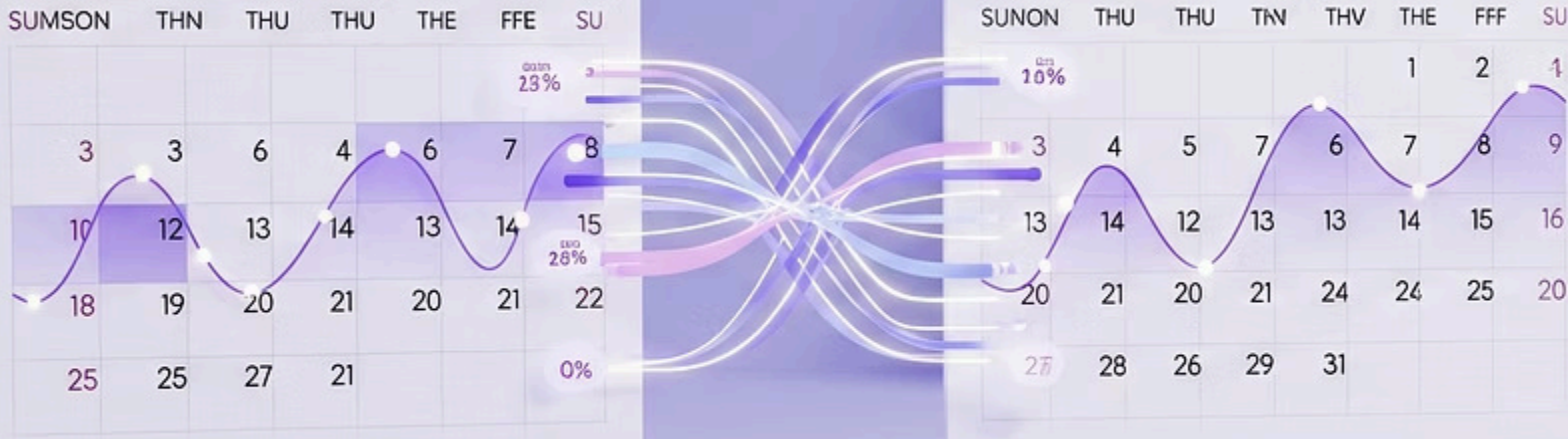
- Chi-squared test
- Population Stability Index (PSI)

Embeddings

- Cosine distance of centroids
- Fréchet Inception Distance (FID-style stats)
- Earth Mover's Distance (EMD)

LLM-specific

- Output length distribution
- Refusal rate changes
- Safety policy trigger rate
- RAG retrieval hit-rate



Windows & Baselines



Reference Window

Gold standard baseline from last stable 30-90 days of production data



Comparison Window

Latest N hours/days of incoming data to check against reference



Advanced Windows

Sliding + seasonal windows accounting for weekday patterns and seasonal trends

Segment by **region/device/user-tier** to effectively localize issues and prevent false alarms

Pipeline Architecture (High-Level)



01

Collect

Features, predictions, and labels (when available)

03

Store

Metrics in TSDB (Prometheus), reports in object storage

05

Act

Retraining triggers, canary/shadow deployments, rollbacks

02

Compute

Batch jobs + streaming statistics calculation

04

Alert

SLO-backed thresholds → PagerDuty/Tickets

Batch Drift (Daily) with Airflow

A pseudo-DAG implementation of daily drift monitoring:

```
@dag(schedule="@daily", catchup=False, tags=["drift"])
def drift_daily():
    ref = load_reference("s3://bucket/gold/2024-06/*")
    cur = load_current("s3://bucket/gold/{{ ds }}/*")

    report = compute_evidently_report(ref, cur)
    store_report(report, path=f"s3://bucket/reports/{{ ds }}/drift.html")

    metrics = extract_alert_metrics(report) # per-feature PSI/KS etc.
    push_to_prometheus(metrics) # pushgateway

    gate = evaluate_thresholds(metrics, policy="strict")
    maybe_page(gate)
    maybe_open_retrain_pr(gate)

drift_daily()
```

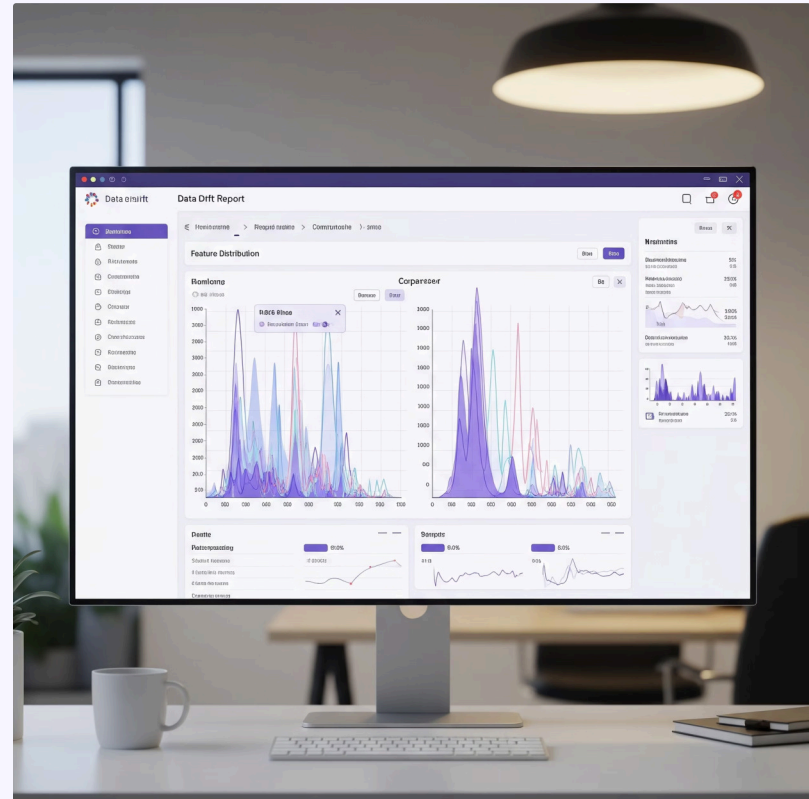
Batch Code Snippet (Evidently)

```
import pandas as pd
from evidently.report import Report
from evidently.metric_preset import DataDriftPreset

# Load parquet/csv snapshots
ref = pd.read_parquet("ref.parquet")
cur = pd.read_parquet("cur.parquet")

# Create report with drift metrics
report = Report(metrics=[DataDriftPreset()])
report.run(reference_data=ref, current_data=cur)
report.save_html("drift_report.html")

# Extract metrics for alerting
summary = report.as_dict()
fmetrics = {
    m["column_name"]: m["metrics"].get("psi") or
                     m["metrics"].get("p_value")
    for m in summary["metrics"][0]["result"]["drift_by_columns"]
}
```



Evidently generates comprehensive HTML reports with distribution visualizations and statistical tests for each feature

Streaming Drift (Minutes) with Kafka + River

```
from river.drift import ADWIN

adwins = {
    "ctr_30d": ADWIN(delta=1e-3),
    "views_7d": ADWIN(delta=1e-3)
}

def on_event(x): # x is a dict of feature values
    alerts = []
    for f, v in x.items():
        if f in adwins:
            before = adwins[f].change_detected
            adwins[f].update(float(v))
            if adwins[f].change_detected and not before:
                alerts.append(f)

    if alerts:
        publish("drift-alerts", {"features": alerts, "ts": now()})
```

Key advantages: River's ADWIN/CUSUM/PHT algorithms provide **online** concept/data drift detection with minimal memory footprint. Use Flink or Spark Streaming for keyed windows and aggregation at scale.

Feature Store Integration

Inference Logging

Log **feature vectors** + **prediction** + **model version** for every inference request

Contract Enforcement

Enforce **feature contracts** (schema & value ranges); reject bad payloads

Reference Snapshots

Materialize "reference" data snapshots from Feature Store for batch drift

Privacy-Safe Auditing

Retain **hashed inputs** (PII-safe) for joinability & compliance audits



Thresholds & Alerting Strategy

Starting Point

- Statistical tests ($p < 0.01$)
- Then calibrate to business impact
- Track false positive/negative rates

Advanced Controls

- **Hysteresis:** Require sustained drift
- **Cool-downs:** Prevent alert storms
- Segment-specific thresholds



Two-Tier Alert System

Page (P1): High PSI/KS across top features + KPI dip

Ticket (P2): Single feature mild drift for 24h





Actions: From Alert → Mitigation



Auto-retrain

Trigger model retraining pipeline (if labels available) with checkpoint guard



Traffic Shift

Gradually route traffic to canary model trained on latest data



Feature Hotfix

Cap, clip, impute; rollback feature versions in Feature Store



Rollback

Switch to last stable model via registry flag when issues severe

CI/CD Guardrails for Drift

Shadow Deployment

Deploy new model in shadow mode, receiving production traffic copies without affecting users

Drift Analysis

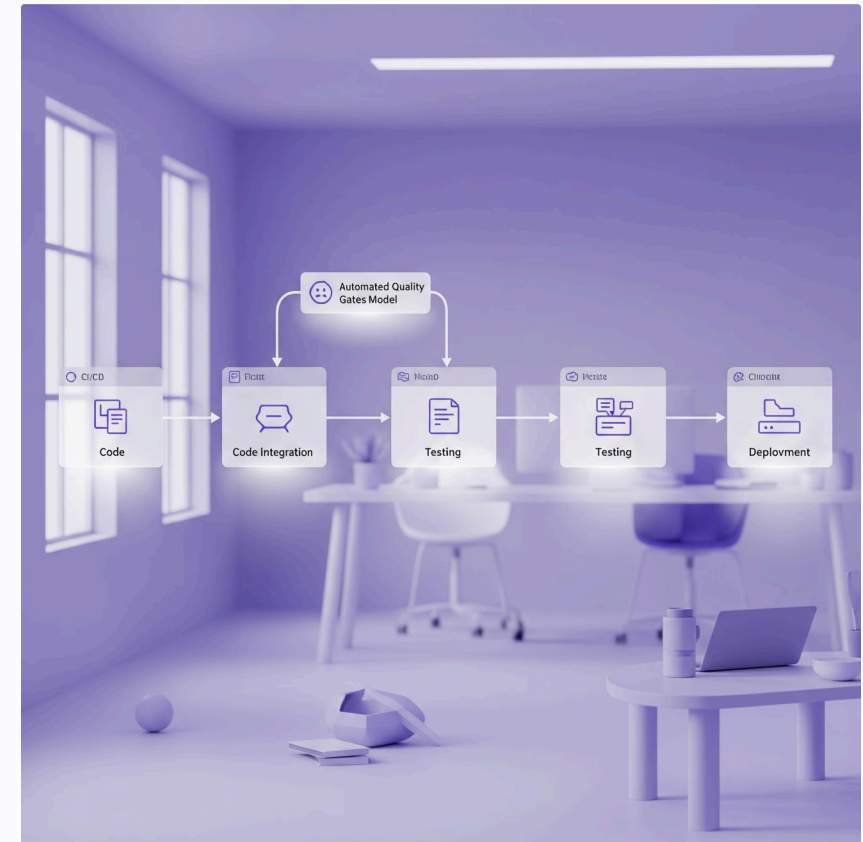
Block promotion if **drift** > **threshold** in shadow traffic compared to current model

Canary Evaluation

Analyze p95 latency, error rates, **win-rate vs baseline**, and drift score

Promotion Decision

Require manual approval when drift is "intentional" (new markets, features, etc.)





LLM/RAG-Specific Drift



Prompt Mix Drift

Domain/user segments shift (track prompt taxonomy over time)

Monitor prompt type distribution and detect when usage patterns change



Retrieval Drift

Key metrics: hit-rate ↓, avg recall@k ↓, document freshness age ↑

Track when retrieval performance degrades or document relevance changes



Output Drift

Toxicity/refusal/length distribution changes

Monitor when generated text characteristics shift unexpectedly



Embedding Drift

Centroid/covariance of response embeddings shift

Track semantic space movement in embeddings over time

Dashboards You Need



Drift Overview

Overall drift score, top impacted features, affected segments

Feature Detail

PSI/KS over time, histograms comparing reference vs current

LLM Quality

Win-rate vs baseline, safety rate, RAG hit-rate trends

Action Log

Retrains, rollbacks, canary outcomes with timestamps

Link dashboards to runbooks for quick resolution during incidents

Governance, Privacy, Compliance

PII Protection

Store **metrics not raw PII**; hash or tokenize sensitive inputs

Access Controls

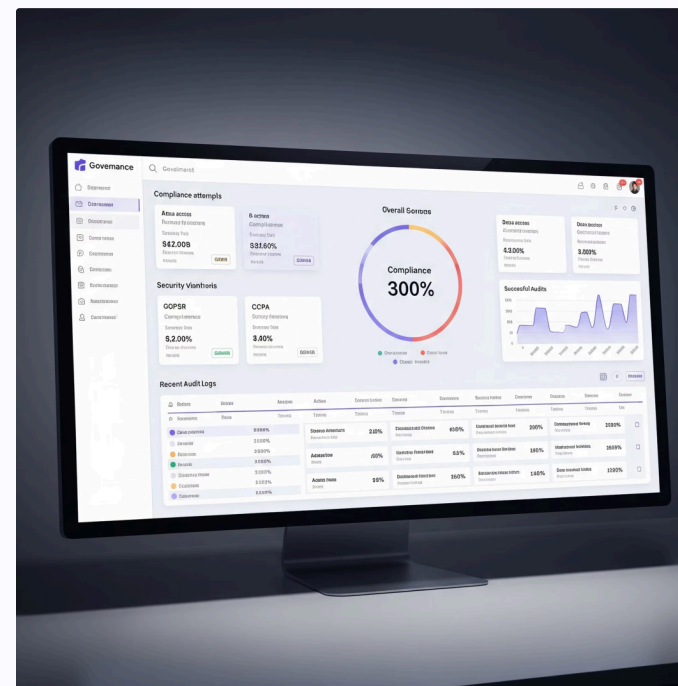
Strict permissions on drift data & reports with role-based access

Audit Trail

Immutable record of when warnings fired, who approved actions

Retention Policy

Keep sufficient history to prove due diligence for audits



- ⊗ Regulated industries (finance, healthcare) require stricter governance and longer retention periods to demonstrate model monitoring compliance

Cost & Performance Tips



Strategic Sampling

Sample large streams intelligently; stratify by key segments to maintain representativeness while reducing compute



Computation Tiering

Compute heavy statistics **offline**; use lightweight detectors for online/real-time monitoring



Smart Caching

Cache histograms & reuse **count-min sketches** for streaming categoricals to minimize memory usage



Focused Alerting

Alert only on **top-k** business-critical features to reduce noise and focus attention where it matters



Implementation Checklist

1 Define Windows

Reference & comparison windows per segment

2 Select Metrics

Choose appropriate metrics per feature type (PSI/KS/JS/ADWIN)

3 Build Pipelines

Batch DAG + Streaming job wired to **Prometheus**

4 Configure Alerts

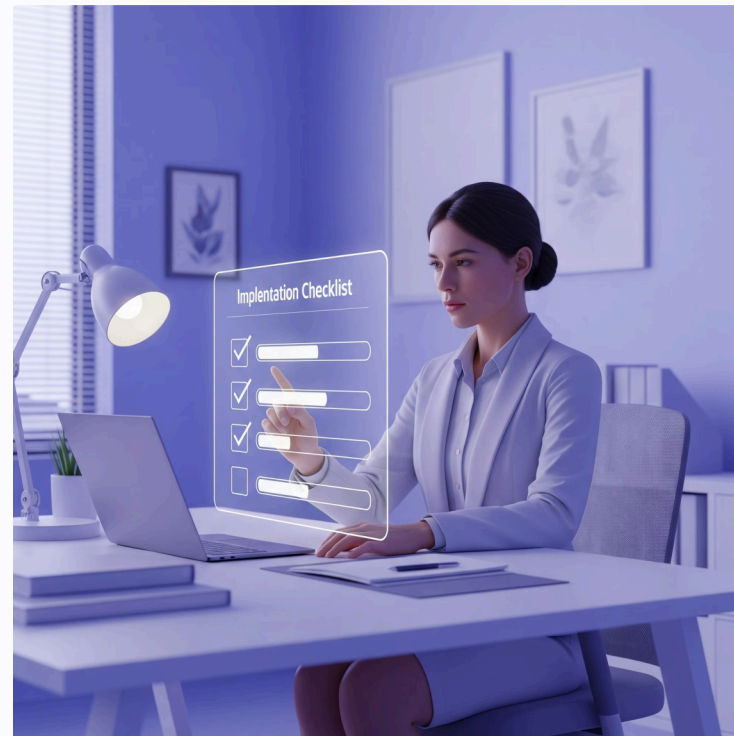
SLOs & alert policies + runbooks for on-call response

5 Automate Actions

Retrain/canary/rollback automation triggered by alerts

6 Create Visibility

Dashboards + audit logging live and accessible



- ❑ Start small and iterate! Begin with batch detection on critical models before expanding to streaming and full automation

Discussion Prompt

Would you gate **production promotions** on drift alone, or require **drift + KPI deltas**?

Drift-Only Approach

- Faster to implement
- More conservative/safer
- Prevents "silent failures"
- Works when KPIs have long feedback loops

Drift + KPI Approach

- Reduces false positives
- Focuses on business impact
- Allows beneficial drift (improving models)
- Creates stronger case for model rollbacks

What approach does your organization take? Why?